GIDFINE LOAN PLATFORM

PRE-PHASE 4 WORKTREE RECONCILIATION & VALIDATION AUDIT REPORT

Target Branch: development | Commit HEAD: afb4323 | Target Stack: React 19 + TypeScript + Convex

Date: October 21, 2026	**Expected HEAD:** afb43239c112864649e4c63f72d74f8b49daebae
Branch Status: Synced with origin/development	**main Status:** Untouched (at f7ed8d1)
Validation Status: 100% Passing (30 TS/Node + 6 Py + Build)	**Preservation Plan:** Checkpoint ready for staging approval

1. Executive Summary & Reconciliation Objective

Prior to initiating Phase 4A (Financial Ledger Schema & Accounting Foundation), a complete, non-destructive inventory and validation audit of all Phase 1–3C modifications was performed. The working tree was verified to strictly adhere to Gidfine safety rules: remaining entirely on branch development, preserving all Python/FastAPI legacy components, modifying zero files on main, and executing no destructive Git operations (no reset, clean, checkout-overwriting, or rebase).

Key Finding: The local working tree contains **7 modified tracked files**, **10 untracked source/test files**, and **11 migration audit artifacts**. All existing test suites pass with 0 failures, TypeScript emits 0 errors, and the Vite production build succeeds cleanly.

2. Repository Status & Inventory Commands

The exact output from Git diagnostic commands executed on the repository checkout:

Git Command	Summary of Command Results
`git status --short`	7 modified tracked files (M), 10 source/test untracked files (??), and report artifacts.
`git diff --stat`	7 files changed, 209 insertions(+), 39 deletions(-) across tracked Convex schema, UI, and docs.
`git diff --name-only`	Identified all 7 tracked files precisely. No unexpected modifications exist.
`git ls-files --others`	Enumerated all untracked test suites, components, services, and reports.
`git log -10 --oneline`	HEAD: afb4323 (HEAD -> development, origin/development) feat: initialize Convex backend foundation

3. Complete File Inventory (Phase 1 through Phase 3C)

File Path	Status	Scope / Functionality
`frontend/convex/schema.ts`	Tracked (M)	Added applications, lenderProposals, consents tables & compound indexes.
`frontend/convex/lib/auth.ts`	Tracked (M)	Production role authorization guards via ctx.auth.getUserIdentity().
`frontend/convex/_generated/api.d.ts`	Tracked (M)	Registered loans and auth modules in Convex fullApi type map.
`frontend/src/lib/convex.tsx`	Tracked (M)	Added useOptionalConvex() hook and client provider fallback.
`frontend/src/components/AdminDashboard.tsx`	Tracked (M)	Added Agency Review Queue and Lender Portal navigation tabs.
`frontend/src/components/.../LoanApplication.tsx`	Tracked (M)	Integrated BorrowerLoanManager with wizard simulation fallback.
`migration/README.md`	Tracked (M)	Index documentation tracking Phase 1-4 deliverables.
`frontend/convex/lib/stateMachine.ts`	Untracked (??)	20-status centralized transition engine & SHA-256 fingerprinting.
`frontend/convex/loans.ts`	Untracked (??)	Phase 1-3B backend mutations, queries, redaction, readiness check.
`frontend/src/services/loanService.ts`	Untracked (??)	Client adapter with pesewas integer math and graceful error handling.
`frontend/src/.../BorrowerLoanManager.tsx`	Untracked (??)	Borrower portal UI, application form modal, affirmative consent card.
`frontend/src/components/AgencyReviewQueue.tsx`	Untracked (??)	Agency underwriting queue, quote submission, stage filtering.
`frontend/src/components/LenderPortal.tsx`	Untracked (??)	Lender review interface with redacted PII and decision buttons.
`frontend/tests/stateMachine.test.ts`	Untracked (??)	7 unit tests for state transitions, role enforcement, SHA-256.
`frontend/tests/mutations.test.ts`	Untracked (??)	14 integration tests for mutations, auth, tenant isolation, redaction.
`frontend/tests/loanService.test.ts`	Untracked (??)	7 unit tests for unit conversions, percentage math, error handling.
`frontend/tests/e2eSecurityAudit.test.ts`	Untracked (??)	2 comprehensive E2E tests for 8-step lifecycle & multi-tenant isolation.

4. Verification & Test Execution Results

All verification commands were executed directly against the active worktree checkout:

Verification Suite	Command	Exit Code	Result
TypeScript Compiler	`npx tsc --noEmit`	0	**PASSED (0 type errors)**
Node Test Runner (All Suites)	`node --experimental-strip-types --test tests/*.test.ts`	0	**PASSED (30 / 30 tests)**
Python Backend Regression	`pytest tests/ -q`	0	**PASSED (6 / 6 tests)**
Vite Production Build	`npx vite build`	0	**PASSED (1,845 modules)**

Breakdown of the 30 Passing Node Tests:

• **stateMachine.test.ts (7 tests):** Standard linear progression, unauthorized role rejection, lender approval bypass guard, agency consent bypass guard, disbursement readiness guard without consent snapshot, multi-tenant isolation guard, deterministic SHA-256 fingerprinting.

• **mutations.test.ts (14 tests):** Unauthenticated caller rejection, unauthorized role rejection, positive amount/tenure validation, audit logging, proposal superseding, tenant isolation, affirmative consent recording, lender PII redaction, non-payment initiation readiness check, borrower ownership enforcement, agency review queue access, and lender proposal decision guards.

• **loanService.test.ts (7 tests):** GHS to integer pesewas conversion, pesewas to formatted GHS string, basis points to percentage formatting, ConvexCloudUnavailableError throwing, input validation, affirmative consent validation, and lender decision handling.

• **e2eSecurityAudit.test.ts (2 tests):** Full 8-step loan lifecycle execution from borrower submission to disbursement readiness; strict enforcement of multi-tenant isolation, role authorization, and server-side data redaction.

5. Verified Functionality & Discrepancy Analysis

State Machine & Security Guards: The 20-status state machine engine is fully isolated on the backend. Transitions strictly require verified roles (borrower, agency, lender) and validate the compound agencyId and lenderId relationships. Bypasses are blocked at the engine level.

Redaction & Privacy: Lender views are verified to redact borrower name, email, phone, and agency profit margin. Only essential risk evaluation parameters (requested amount, approved tenure, risk tier, proposed interest rate) are exposed.

Boundary Safety: Disbursement readiness check verifies all preconditions (underwriting approval, lender acceptance, borrower affirmative consent, and SHA-256 fingerprint match) without initiating financial transfers or activating schedules.

Discrepancies / Conflicts: NONE. No conflicting implementations or type mismatches were found. The Python legacy backend remains completely untouched and passing.

6. Safe Proposed Git Preservation Strategy

Per the strict safety requirements, no staging, committing, pushing, or resetting has been executed. To preserve the existing work safely before proceeding to Phase 4A, the following sequential strategy is proposed for approval:

Step 1 — Create a Local Snapshot Branch:
git branch backup/pre-phase4-baseline
Creates an immediate reference pointer to HEAD afb4323 without altering the working tree.

Step 2 — Logical Phase Commits on development:
Commit 1: feat(convex): implement Phase 1 schema, state machine, and audit logging
Commit 2: feat(convex): implement Phase 2 secure mutations, queries, and auth guards
Commit 3: feat(frontend): integrate Phase 3A-C borrower, agency, and lender portals with loan service
Commit 4: test: add comprehensive 30-test suite covering state machine, mutations, and e2e security
Commit 5: docs: add Phase 1-3C reconciliation and security audit reports

Step 3 — Gatekeeper Verification Before Phase 4A:
Run full test suites post-commit to confirm clean status prior to creating the Phase 4A ledger schema.